

Introduction au Langage C

TP n°3 : Groupe A

EPITA Cyber 1
2025-2026

1 Consignes

Le sujet du TP3 est composé de **7 exercices de programmation** en langage C.

Chaque exercice contient l'**objectif de la fonction** à écrire et des **exemples d'exécution**.

Pour chaque exercice, vous devez **soumettre un fichier ".c"** dans le **dépôt associé à l'exercice**.
Testez vos fonctions avec une fonction **main**, et **supprimez la fonction main avant de soumettre**.

→ **Votre fichier ".c" soumis ne doit plus contenir de fonction "main"**.

Les dépôts d'exercices se trouvent sur **Moodle**.

Chaque fichier .c doit commencer par un **bloc de commentaires avec vos informations personnelles** : Nom, prénom, date, et groupe (voir exemple ci-dessous).

Veillez à **déposer la version finale de votre code**.

→ **La soumission de votre code est définitive**.

Consignes additionnelles :

- Le nom du fichier .c doit être **exercice[i].c**, où [i] est à remplacer par le numéro de l'exercice, par exemple **exercice1.c** pour l'exercice numéro 1.
- Votre programme doit compiler avec la commande :
\$gcc -std=c11 -Wall -Wextra -Werror exercice1.c -o exercice1
- Respectez les prototypes des fonctions données dans l'énoncé.
- Les bibliothèques autorisées sont : stdio.h.

Exemple : Un fichier contenant un bloc d'informations personnelles.

```
1  /*  Nom: Barrot Nathanael
2      Date: 06/10
3      Groupe: A
4  */
5
6  #include <stdio.h>                                // import de stdio.h
7
8  int main(void)
9  {
10     int num=0;                                       // definition de num pour
11     stockage                                         // affichage dans le terminal
12     printf("Entrer un entier: ");                  // "%d" pour entrer un entier
13     scanf("%d", &num);
14     return 0;
15 }
```

2 Exercice n°1 : Somme

Objectif :

Ecrire une fonction **sumTab** qui prend en argument un tableau d'entier *tab* et sa taille *n*, et **retourne** la somme des éléments du tableau.

Le prototype de la fonction doit être :

```
1 int sumTab(int tab[], int n);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `sumTab(tab, 3)` retourne 6.
- Avec $tab = \{5, 5, 5, 5, 5\}$, `sumTab(tab, 5)` retourne 25.
- Avec $tab = \{0, 0, 7\}$, `sumTab(tab, 3)` retourne 7.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

3 Exercice n°2 : Recherche Valeur

Objectif :

Ecrire une fonction **searchTab** qui prend en argument un tableau d'entier *tab*, sa taille *n*, et un entier *value*, et **retourne** 1 si l'entier *value* se trouve dans le tableau et 0 sinon.

Le prototype de la fonction doit être :

```
1 int searchTab(int tab[], int n, int value);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `searchTab(tab, 3, 2)` retourne 1.
- Avec $tab = \{5, 5, 5, 5, 5\}$, `searchTab(tab, 5, 19)` retourne 0.
- Avec $tab = \{7, 7, 7\}$, `searchTab(tab, 3, 7)` retourne 1.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

4 Exercice n°3 : Minimum

Objectif :

Ecrire une fonction **minTab** qui prend en argument un tableau d'entier *tab* et sa taille *n*, et **retourne l'indice** de la valeur minium du tableau.

Le prototype de la fonction doit être :

```
1 int minTab(int tab[], int n);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `minTab(tab, 3)` retourne 0.
- Avec $tab = \{10, 5, -5, 5, 10\}$, `minTab(tab, 5)` retourne 2.
- Avec $tab = \{7, 7, 7\}$, `minTab(tab, 3)` retourne 0.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

5 Exercice n°4 : Swapper

Objectif :

Ecrire une fonction **swapValue** qui prend en argument un tableau d'entier *tab*, sa taille *n*, et deux entier *i* et *j*, et **échange** les valeurs du tableau des cases d'indices *i* et *j*, puis **affiche dans le terminal** le tableau.

Le prototype de la fonction doit être :

```
1 void swapValue(int tab[], int n, int i, int j);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `swapValue(tab, 3, 0, 2)` affiche dans le terminal :

```
1 3 2 1
```

retourne $tab = \{3, 2, 1\}$.

- Avec $tab = \{10, 0, 5, -5\}$, `swapValue(tab, 4, 0, 3)` affiche dans le terminal :

```
1 -5 0 5 10
```

- Avec $tab = \{1, 2, 3, 1, 3, 2, 4\}$, `swapValue(tab, 7, 3, 6)` affiche dans le terminal :

```
1 1 2 3 4 3 2 1
```

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

6 Exercice n°5 : Reverse

Objectif :

Ecrire une fonction **reverseTab** qui prend en argument un tableau d'entier *tab* et sa taille *n*, et inverse l'ordre des valeurs de *tab*, et **affiche dans le terminal** le tableau inversé.

Le prototype de la fonction doit être :

```
1 void reverseTab(int tab[], int n);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `reverseTab(tab, 3)` affiche dans le terminal :

```
1 3 2 1
```

- Avec $tab = \{10, 0, 5, -5\}$, `reverseTab(tab, 4)` affiche dans le terminal :

```
1 -5 5 0 10
```

- Avec $tab = \{1, 2, 3, 1, 3, 8, 4\}$, `reverseTab(tab, 7)` affiche dans le terminal :

```
1 4 8 3 1 3 2 1
```

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

7 Exercice n°6 : Compter les Caractères

Objectif :

Ecrire une fonction **countChar** qui prend en argument un tableau de caractères *tab*, sa taille *n*, et un caractère *lettre*, et **retourne** le nombre de fois où le caractère *lettre* apparaît dans le tableau.

Le prototype de la fonction doit être :

```
1 int countChar(char tab[], int n, char lettre);
```

Exemples d'appels :

- Avec `tab = {'M', 'o', 't'}`, `countChar(tab, 3, 'o')` retourne 1.
- Avec `tab = {'M', 'i', 'n', 'i', 'm', 'u', 'm'}`, `countChar(tab, 7, 'm')` retourne 2.
- Avec `tab = {'M', 'a', 'x', 'i', 'M', 'u', 'M'}`, `countChar(tab, 7, 'm')` retourne 0.
- Avec `tab = {'M', 'a', 'x', 'i', 'm', 'u', 'm'}`, `countChar(tab, 7, 'z')` retourne 0.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

8 Exercice n°7 : Compter les Lettres

Objectif :

Ecrire une fonction **countLetter** qui prend en argument un tableau de caractères *tab*, sa taille *n*, et un caractère *lettre*, et **retourne** le nombre de fois où la lettre *lettre* apparaît dans le tableau, sous forme sa forme miniscile ou majuscule.

Le prototype de la fonction doit être :

```
1 int countLettre(char tab[], int n, char lettre);
```

Exemples d'appels :

- Avec $\text{tab} = \{'M', 'o', 't'\}$, `countChar(tab, 3, 'o')` retourne 1.
- Avec $\text{tab} = \{'M', 'i', 'n', 'i', 'm', 'u', 'm'\}$, `countChar(tab, 7, 'm')` retourne 3.
- Avec $\text{tab} = \{'M', 'a', 'x', 'i', 'M', 'u', 'M'\}$, `countChar(tab, 7, 'm')` retourne 3.
- Avec $\text{tab} = \{'M', 'a', 'x', 'i', 'm', 'u', 'm'\}$, `countChar(tab, 7, 'z')` retourne 0.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.